

Block 2

Generate a Minimal Define-XML v2.1

15 min lecture • 20 min exercise • 5 min review

Why not hand-roll it?

Writing Define-XML by hand means you personally own:

- **Four namespaces** — `def:`, `xlink:`, `xsi:`, and the ODM default
- **Strict element order** — children in the sequence the schema demands
- **Dozens of required attributes** — per element, per version
- **Cross-references** — every OID pointing at something real

One typo ships a file that fails **silently**, weeks later, at a gateway.

“ odmlib moves all four into the model.

Hand-rolled failures are silent; **odmlib failures are loud.**

”

What you're building

```
ODM (FileOID, FileType, CreationDateTime, ODMVersion, def:Context)
└ Study (OID)                                     ← assign
  └ GlobalVariables (StudyName, StudyDescription, ProtocolName)
    └ MetaDataVersion (OID, Name, def:DefineVersion) ← assign
      └ def:Standards → Standard (SDTMIG 3.4)
        └ ItemGroupDef "DM" [list]
          └ Description
            └ ItemRef × 4      schema order matters
            └ def:Class
              └ def:leaf ——— ID ↔ ArchiveLocationID
        └ ItemDef × 4 (Description, CodeListRef, Origin) [list]
        └ CodeList "CL.SEX" → CodeListItem F / M [list]
```

Five construction rules

1. **Required attributes are enforced at `__init__`** – omit `Name` and you get `OdmlibRequiredAttributeError` *now*, not a broken file three steps later
2. **Element text is `_content=`** – `DEF.TranslatedText(_content="Age", lang="en")`
3. **Single children are assigned, list children are appended**
`item.Description = DEF.Description()` vs `igd.ItemRef.append(...)`
4. **Namespace prefixes are invisible** – write `Structure=`, `href=`;
the model puts `def:` / `xlink:` back on serialization
5. **Required child *elements* are not checked at construction** – a dataset missing its `def:leaf` surfaces at *validation* (Block 3)

Getting it back out

Call	Use it for
<code>odm.write_xml(path)</code>	The document to a file, with XML declaration
<code>elem.to_xml_string()</code>	Any element as a string – debugging and diffs
<code>elem.to_element()</code>	A real <code>ElementTree.Element</code> – interop (<i>new in 0.2.1</i>)
<code>odm.write_json(path)</code> / <code>to_dict()</code>	Define-XML content as JSON / dicts

Human-readable view, one command (*demo in segment 7*):

```
python -m defineutils.definehtml -d output/define_dm.xml -o define_dm.html
```

Watch out for these

- `study.MetadataVersion.append(...)` — no: single object, **assign** it
- Forgetting `Context` on `DEF.ODM(...)` — `def:Context` is required in v2.1 *(and is the deliberate error in one of Block 3's broken files)*
- `ArchiveLocationID` **must match a leaf ID** — a mismatch is a ref/def error the OLD checker catches
- Adding children **out of schema order** — harmless to `write_xml()`, but `validate()` flags it; `reorder_object()` repairs it
- Hand-building an XML string for "just one small element" — `to_xml_string()` **exists**; hand-rolled markup is how namespace bugs get in

Your turn → `create_define_exercise.ipynb` — **4 TODOs, ~20 min**